why ten?

prof. dr. nicolas meseth

1. the place value trick
2. why two?
3. reading bytes
4. colours as digits

part 1

the place value trick

why do we count to ten?

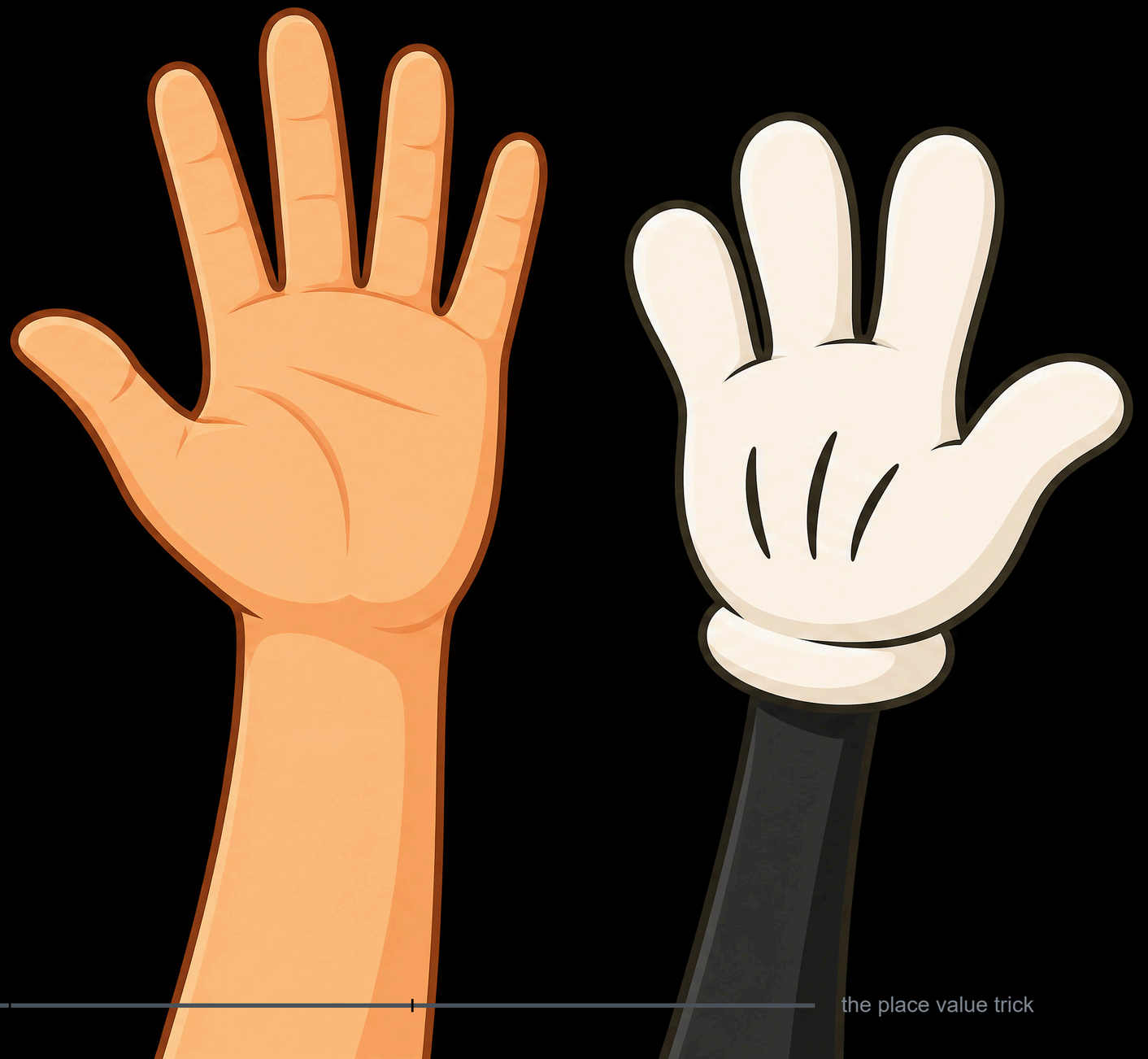
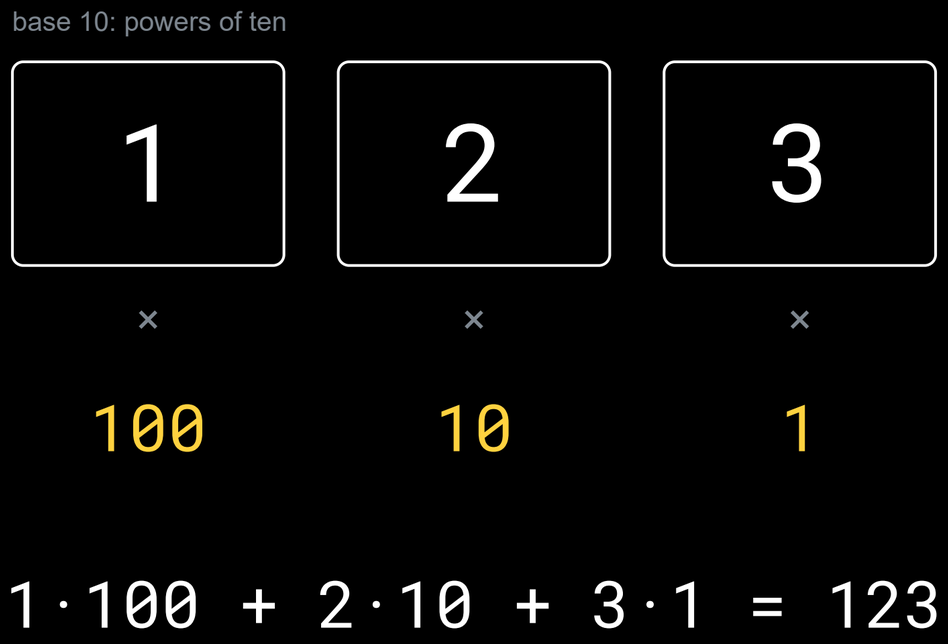


image: ai-generated (gpt-image-2)

what 123 really says

base 10: powers of ten

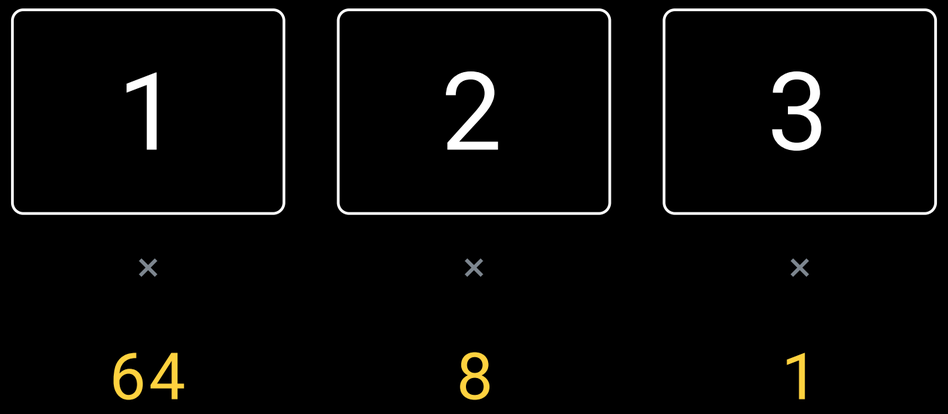




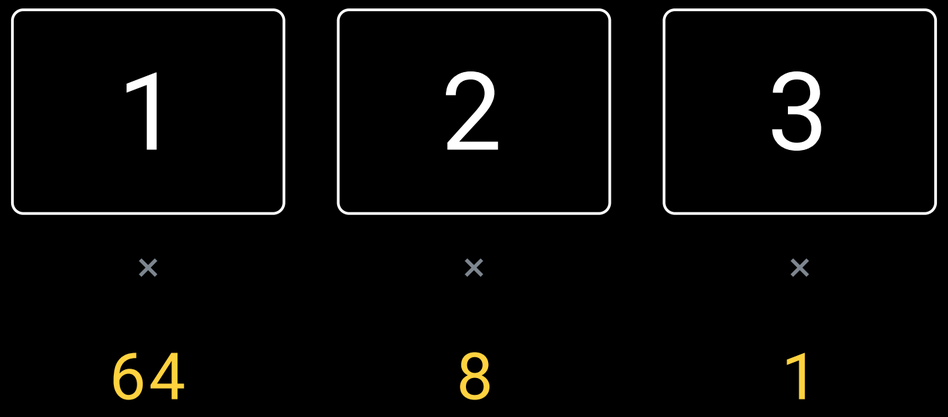
every place carries a power of ten.

the eight-finger creature writes 123

base 8: powers of eight



base 8: powers of eight



$$1 \cdot 64 + 2 \cdot 8 + 3 \cdot 1 = 83$$

the same recipe with another base gives another number.

and a creature with two flippers?



image: ai-generated (gpt-image-2)

two flippers: counting in binary

we write	the dolphin writes
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
8	1000

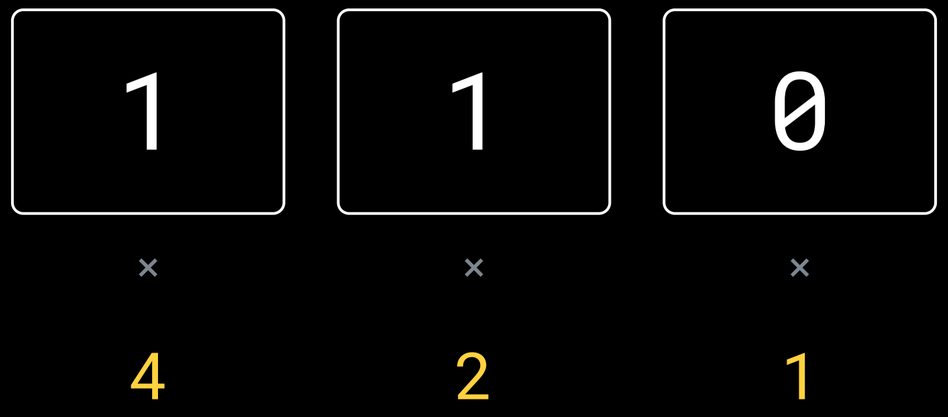
a new place opens
when the digits run out,
just like 9 to 10

the dolphin writes 110

base 2: powers of two



base 2: powers of two



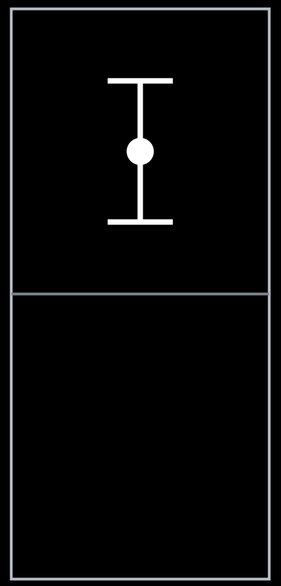
$$1 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 6$$

nothing new: the third time is the same trick.

part 2

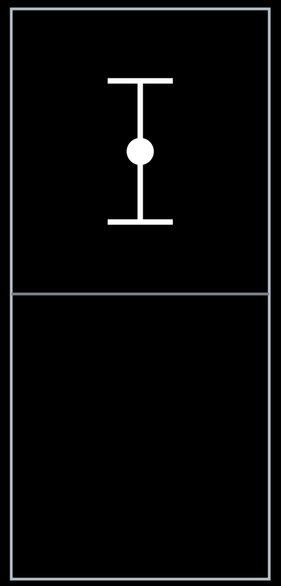
why two?

why two, of all things?

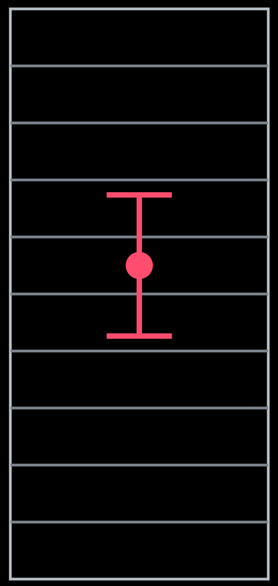


2 states

why two, of all things?



2 states



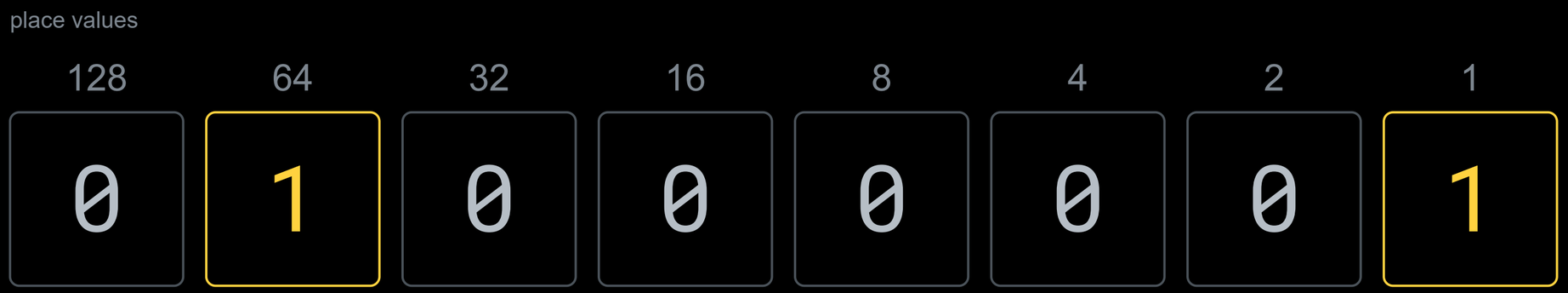
10 states

the same wobble on the wire.
with two states it never matters,
with ten it always does.

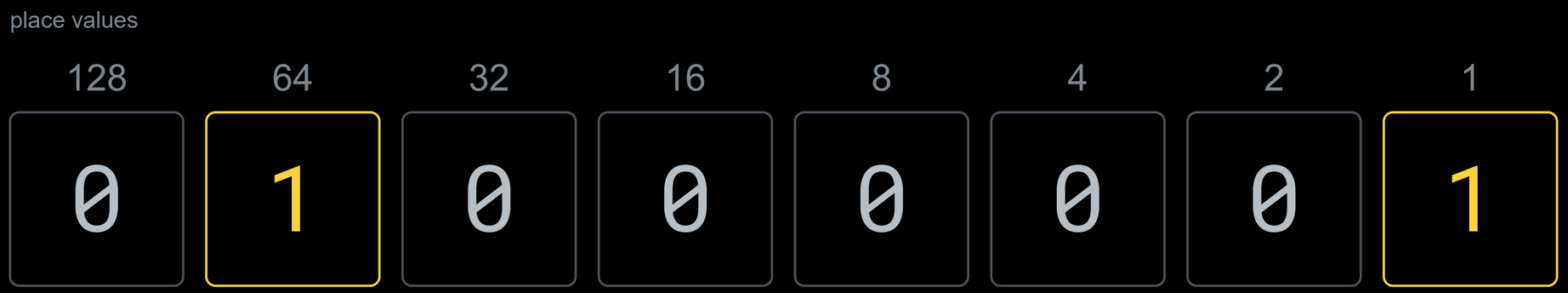
the price: more places.
places are cheap.

two states are the cheapest thing you can build reliably.

eight bits make a byte



eight bits make a byte



$$64 + 1 = 65$$

$2^8 = 256$ values, 0 to 255

256 values, and every one of them is a place value sum.

the byte switchboard on hands-on computer science, code 5470

which switch changes the number the most?
how do you set 255, and how do you set 1?
what happens when you count one step past 255?

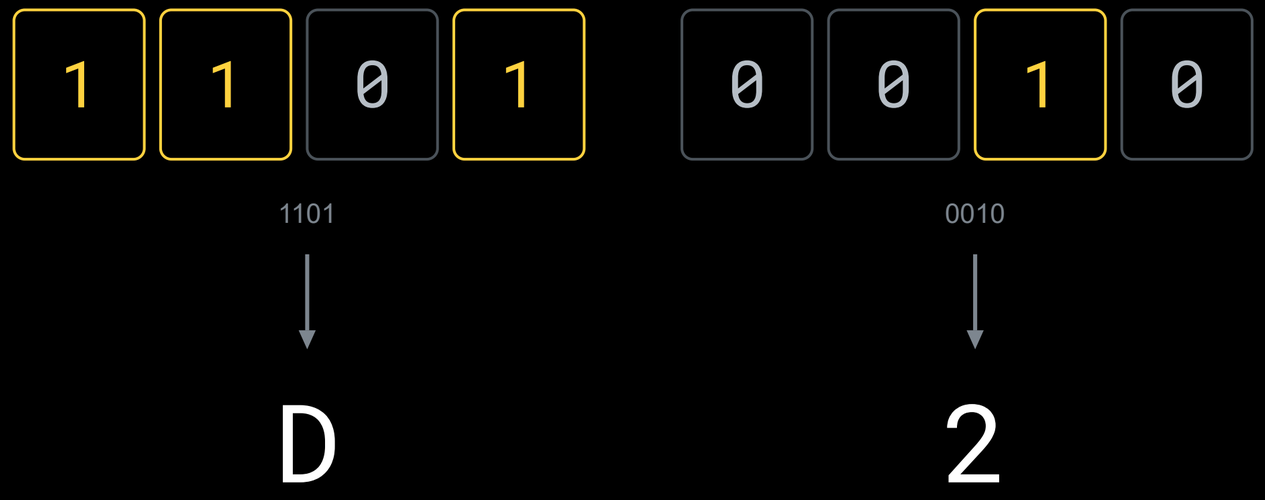
part 3

reading bytes

hex: four bits at a time



hex: four bits at a time



four bits have 16 states.
so one hex digit fits a nibble,
and a byte fits two.

hex: four bits at a time



1101 → D

0010 → 2

four bits have 16 states.
so one hex digit fits a nibble,
and a byte fits two.

ten digits are not enough, so six letters help out

0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

hex is not a new number world, only a reading aid.

the same file, twice

the bytes, in hex	the same bytes as characters
`89 50 4E 47 0D 0A 1A 0A`	`.PNG....`
`00 00 00 0D 49 48 44 52`	`....IHDR`

the same file, twice

the bytes, in hex	the same bytes as characters
89 50 4E 47 0D 0A 1A 0A	.PNG....
00 00 00 0D 49 48 44 52	....IHDR

every png file starts like this: the first bytes say what kind of file it is.
a dot stands for a byte that is not a printable character.

two ladders of units

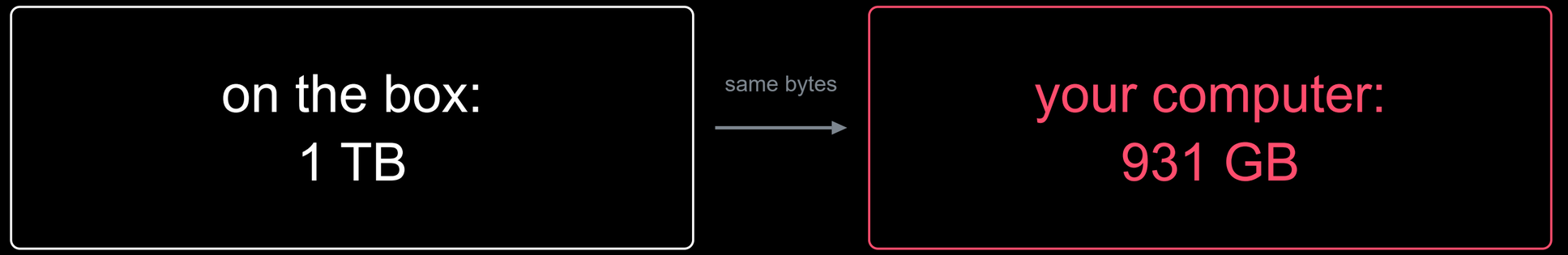
decimal: each step × 1000		binary: each step × 1024	
byte	1		1
kilobyte	1 000		1 024
megabyte	1 000 000		1 048 576
gigabyte	1 000 000 000		1 073 741 824
terabyte	1 000 000 000 000		1 099 511 627 776

both are called the same in everyday speech. that is where the trouble starts.

on the box:
1 TB

`1 000 000 000 000 bytes`

counted up the decimal ladder, × 1000



`1 000 000 000 000 bytes`

counted up the decimal ladder, × 1000

`1 000 000 000 000 ÷ 1 073 741 824`

counted down the binary ladder, ÷ 1024^3

nobody cheated: there are two ladders and one word.

part 4

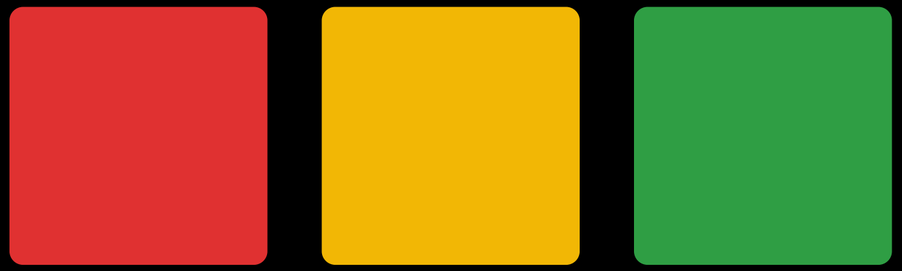
colours as digits

colours can be digits too

the agreed digits

- 0
- 1
- 2
- 3

one character, three colour positions



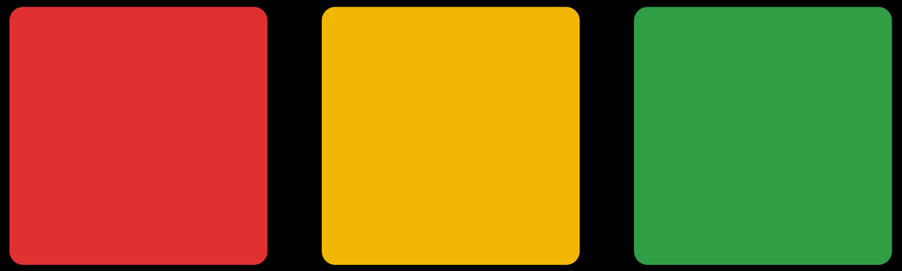
$\times 4^2$ $\times 4^1$ $\times 4^0$

colours can be digits too

the agreed digits

- 0
- 1
- 2
- 3

one character, three colour positions

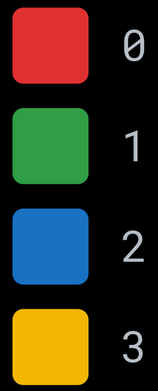


× 4^2 × 4^1 × 4^0

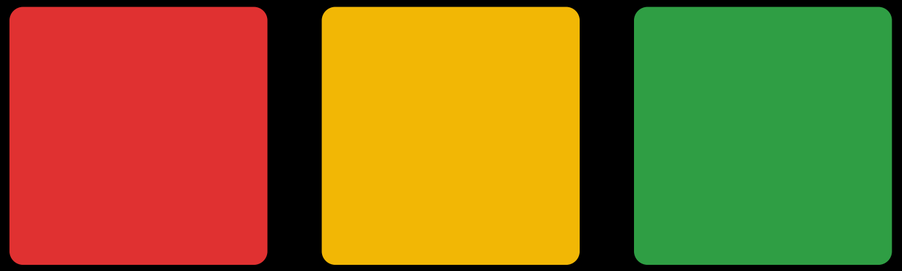
$$0 \cdot 16 + 3 \cdot 4 + 1 \cdot 1 = 13$$

colours can be digits too

the agreed digits



one character, three colour positions



$\times 4^2$ $\times 4^1$ $\times 4^0$

$$0 \cdot 16 + 3 \cdot 4 + 1 \cdot 1 = 13$$

$4^3 = 64$ combinations, enough for an alphabet

k colours are k digits, and position matters.

ten was never special.

ten was never special.

it is the number of fingers on two hands, and nothing else.